



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



Practical Aim:

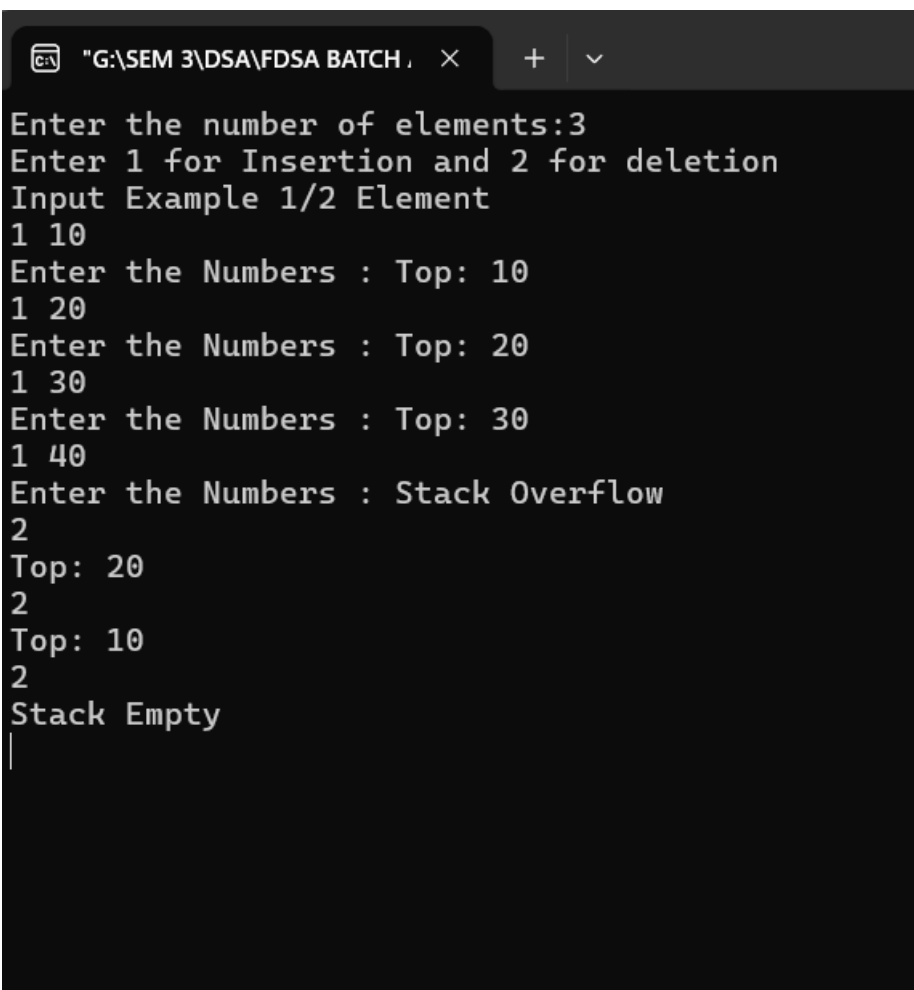
To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

Problem: A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed. Describe how your solution tracks whether the stack is full or empty.

What are the exact conditions you check before a push and before a pop, and what does your program do when those conditions fail?

For Solution: <https://github.com/buildwithneel/FDSA-BATCH-ARP-/tree/main/Practical%206>

Output:



```
"G:\SEM 3\DSA\FDSA BATCH" × + ▾
Enter the number of elements:3
Enter 1 for Insertion and 2 for deletion
Input Example 1/2 Element
1 10
Enter the Numbers : Top: 10
1 20
Enter the Numbers : Top: 20
1 30
Enter the Numbers : Top: 30
1 40
Enter the Numbers : Stack Overflow
2
Top: 20
2
Top: 10
2
Stack Empty
|
```

Problem Faced: The main difficulty was managing the fixed-size stack using an array and correctly checking whether the stack was full or empty before performing push and pop operations.

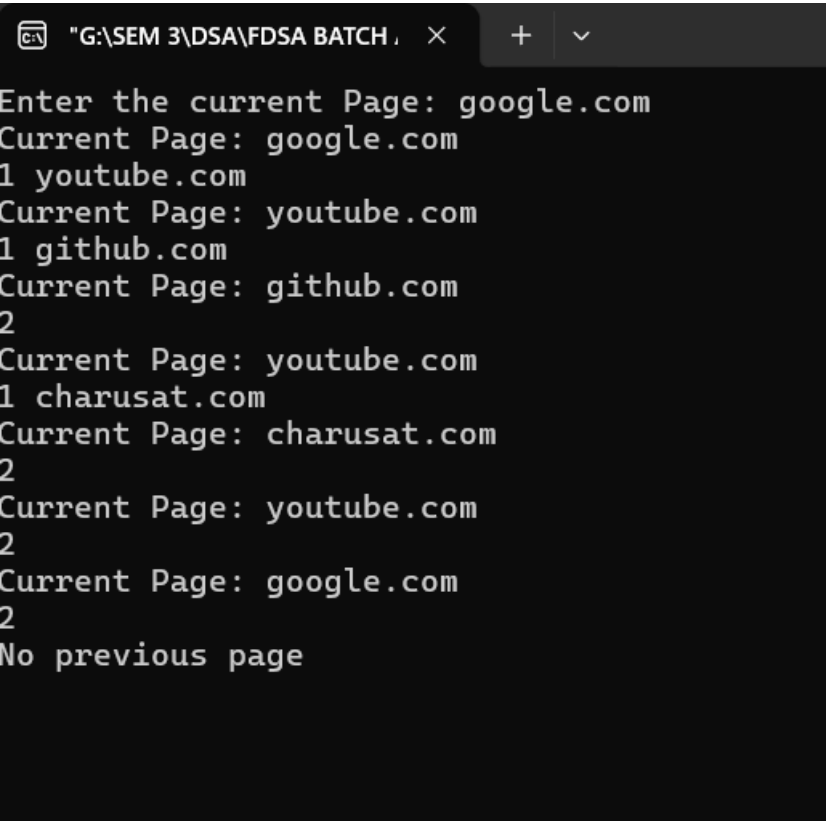
Conclusion: The fixed-capacity stack was successfully implemented using an array and a `top` variable. Overflow and underflow conditions were handled correctly.

Problem: A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation.

Describe how your solution differs from Problem 1 in how it handles memory. What happens in your solution when the user presses back on the very first page with no history left?

For Solution: <https://github.com/buildwithneel/FDSA-BATCH-ARP-/tree/main/Practical%206>

Output:

A screenshot of a terminal window with a dark background. The window title bar shows a file path "G:\SEM 3\DSA\FDSA BATCH" and standard window controls. The terminal displays a sequence of commands and outputs for a browser history simulation. The commands are: "Enter the current Page:", "1 youtube.com", "1 github.com", "2", "1 charusat.com", "2", "2", and "2". The corresponding outputs are: "Current Page: google.com", "Current Page: youtube.com", "Current Page: github.com", "Current Page: youtube.com", "Current Page: charusat.com", "Current Page: youtube.com", "Current Page: google.com", and "No previous page".

```
"G:\SEM 3\DSA\FDSA BATCH" × + v
Enter the current Page: google.com
Current Page: google.com
1 youtube.com
Current Page: youtube.com
1 github.com
Current Page: github.com
2
Current Page: youtube.com
1 charusat.com
Current Page: charusat.com
2
Current Page: youtube.com
2
Current Page: google.com
2
No previous page
```

Problem Faced: The main difficulty was managing the browser history and ensuring that the user could not go back when only the first page was remaining.

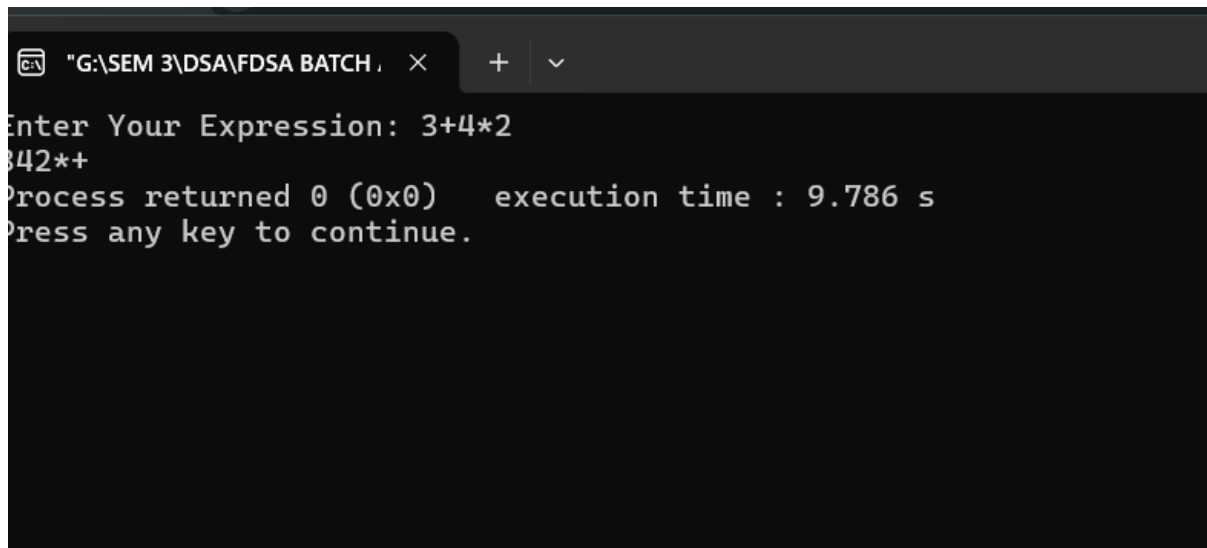
Conclusion: The browser history was successfully implemented using an array-based stack. New pages were added using push operations and the back operation removed the current page using pop logic.

Problem: A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. $3 + 4 * 2$ or $(3 + 4) * 2$). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

Describe the role the stack plays in your conversion. What does your solution do when it encounters a closing bracket, and what happens to the operators that were waiting on the stack at that point?

For Solution: <https://github.com/buildwithneel/FDSA-BATCH-A-ARP-/tree/main/Practical%206>

Output:



```
"G:\SEM 3\DSA\FDSA BATCH" x + v
Enter Your Expression: 3+4*2
342*+
Process returned 0 (0x0) execution time : 9.786 s
Press any key to continue.
```

Problem Faced: The main difficulty was handling operator precedence and brackets correctly while storing operators in the stack and generating the postfix expression.

Conclusion: The infix expression was successfully converted into postfix form using an array-based stack. Operators and brackets were processed according to their precedence and the postfix expression was generated correctly.

Key-Question:

1. [Problem 1] A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Ans: In a circular array, the `front` and `rear` pointers should move back to the beginning of the array when they reach the last index. This can be done using modulo operation, such as `rear = (rear + 1) % n` and `front = (front + 1) % n`. The array would be considered full when `(rear + 1) % n == front` (for the common design that keeps one slot empty).

2. [Problem 2] Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Ans: Yes, a linked list stack can still run out of space. Its actual limit depends on the available memory (RAM and memory provided by the operating system). The limit is invisible in the code because nodes are created dynamically using memory allocation, so the program does not define a fixed maximum number of nodes

3.[Problem 3] operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

Ans: If all operators were treated as having equal precedence, the postfix expression would not preserve the correct order of operations. For example, multiplication and division should be performed before addition and subtraction. Treating them equally could place operators in the wrong order and produce an incorrect postfix expression and therefore an incorrect result.